Danmarks Tekniske Universitet

Gruppe 16: Star Tracker 3000™

Mechatronics 41028 report

Author

Janus Meier - s223996
Oscar Sjelle - s224036
Niels Frederik von Tabouillot - s224013
Liou Xia - s224028
Jacob Osvald Beeken Holm - s234413
Elias Valtteri Vähälummukka - s234379
Marcus Dystrup Pedersen - s234392

12. maj 2025

1 Ide og idegenerering

Vi fik en idé om, at det kunne være fedt at lave en prototype, der kunne vise og tracke stjernebilleder. Dette projekt ville opfylde de fleste af kursets krav til, hvad et projekt skal indeholde:

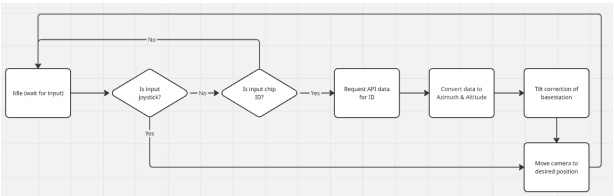
- IoT connectivity (API kald)
- Mekanisk funktionalitet (3D printede konstruktion)
- En aktuator (servo og stepper motor)
- En sensor (IMU, NFC scanner)

Vores idé var, at brugeren giver vores system et input ved brug af NFC eller manuel input. Ud fra inputtet vil systemet skulle vise brugeren, hvor på himlen stjernebilledet var på det givne tidspunkt. Dette skulle ske ved hjælp af to motorer, der fik en vinkel input ud fra informationen omkring stjernebilledet. Formålet med denne idé var at stille flere af disse systemer i parker og naturområder, for at øge læringen og interessen omkring rummet og himmellegemer for børn og unge mennesker. Yderligere kom vi på at erstatte fingeren der peger på stjernebilledet med et GoPro session. Derved har brugeren mulighed for at sætte sit GoPro i og tage timelapse billeder af stjernehimlen i løbet af natten.

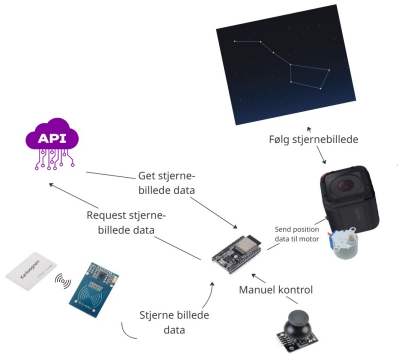
2 Overblik af relevante design elementer

Vores system består af flere mindre dele, som man kan se på Figur 1b, så har vi en RFID-scanner, som vi bruger til at læse kortchips og kort med NFC. Vi har hardcoded nogle forskellige korts UID til forskellige stjernebilleder. Når brugeren scanner et specifikt stjernekort, så sender RFID-scanner UID'et til ESP32'en. Når ESP32'en registrerer et tilhørende UID til et stjernebillede, så ved den, at det for eksempel er Karlsvognen, der er tale om. Derefter så sender den en request til API'en for at få Karlsvognens position. Vores mikrokontroller får, derfra en payload i grader og azimuth. Derefter laver ESP32'en nogle beregninger, da den skal omregne det til vinklen i forhold til dens position. Så sender den positionsdataen til motorerne, så den flytter kameraet. Dette loop mellem API og motorerne sker hvert minut, så kameraet altid er opdateret rigtigt i forhold til positionen på himlen.

Ydermere har vi lavet en 'manual mode'. Hvis brugeren trykker joysticket ned, så overruler det trackingen af stjerne billedet, og brugeren kan nu styre kameraets position helt selv. Dette bliver ved med at ske indtil at brugeren trykker ned på joystick knappen igen.



(a) Flowchart of system

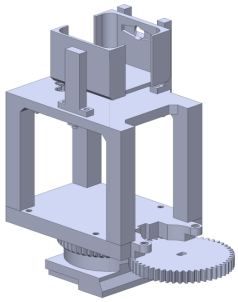


(b) System diagram

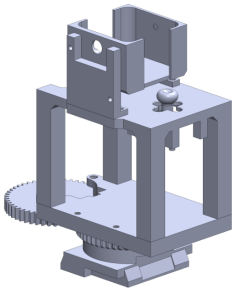
Figur 1: Designoverblik

3 Beskrivelse og visualisering af fysisk prototype

For at have en funktionel stjernetracker skal man have en fysisk model, som kan holde alle ens dele. Modellen i vores prototype skal kunne holde GoPro, NFC-reader, steppermotor, servomotor, joystick, ESP32-breakout board og en stepper-driver. Dette betyder, at vi skal have en kompleks model med monteringshuller til alle de elektroniske komponenter, samt en gearing mellem stepper og stjernetracker for at få en mere blød gearing og en mere kompakt model.



(a) Model af startrackeren forfra



(b) Model af startrackeren bagfra

Figur 2: Startracker

For at holde alle ting har vi en base. Denne base har skruehuller til det øvre stadie og ESP32-breakout boardet. Basen er designet til at rotere på en holder, vi har lavet til en tripod-stand. For at dreje basen monterer vi en steppermotor på basen og har lavet et gear, der mesher med det gear, vi har designet ind i tripod-mount. Denne base sørger for, at alt hænger sammen, og vi kan holde alle ledningerne og komponenterne i en kompakt model.

Det øvre stadie er specielt designet til at holde en servo og et joystick. Dette joystick er placeret lige uden for kameraets rækkevidde, hvilket gør, at kameraet ikke ser joysticket, og

at ens fingre ikke bliver klemt, når det drejer under manuel styring. For at have kameraet monteret og kunne rotere har vi en servo monteret korrekt over det øvre stadie i en præcis afstand, der gør, at vi kan holde modellen kompakt og præcis.

4 Beskrivelse af elektronik

For at kunne tracke en stjerne eller anden type for himmellegemer kræver det at man ved, hvor på himlen legemet er, og hvilken position man har i forhold til dette legeme. Da dette er en alfa-prototype opsætter vi stjerne-trackeren mod nord og hardcoder placerings koordinater for København. For at kende ens placering skal man kende ens latitude og longitude, hvilket korespondere til ens placering på jorden, København har koordinaterne:

$$\text{Latitude} = 55.6761$$

$$\text{Longitude} = 12.5683$$

Efter man kender sine koordinater kan man bruge et kompas til at pege trackeren mod nord, hermed kender man sin vinkel og placering. Nu bruger man et API kald til at få det bestemte legemes azimut og altitude koordinater, hvilket er vinklen i forhold til nord og vinkel fra vandret. Nu prompter man stepper til at dreje til den rigtige Azimut vinkel og servoen til den rette altitude. Hermed har man lavet et system der kan finde et legeme på himlen, dette kan kombineres med en RFID sensor der læser data fra kort eller chips, dette data korrespondere til et legeme der så kan findes på himlen. I videre prototyper ville vi bruge en true North sensor og en GPS til at lokalisere ens vinkel og placering på jorden og dermed ville man kunne bringe stjerne-trackeren over alt på jorden og finde de legemer man gerne vil.

4.1 NFC

Måden brugeren fortæller systemet, hvilket stjerne billede der skal peges på er ved brug af NFC teknologi. Vi har nogle forskellige NFC chips/kort, som vi allerede har loadet ind i systemet. Når brugeren scanner et specifikt kort, det kunne være *Karlsvognen kortet* så ved systemet at det er Karlsvognen den skal hente data fra. Det vil altså sige at hvert chip/korts UID er et bestemt legeme på himlen.

4.2 API

Vi bruger en API lavet af "astronomyapi"til at få altituden og azimut-vinklen. Dette gør vi ved at refere til websites URL streng og kalde en get-kommando, hvori vi får en JSON-objekt, som indeholder informationer om den bestemte stjerne eller stjerne-billedet. Denne URL streng består af:

$$https://api.astronomyapi.com/api/v2/bodies/positions/$$

Hvorefter man indsætter:

```
starName + "?latitude="+ String(latitude) + "&longitude="+ String(longitude)
+ "&elevation="+ String(elevation) + "&from_date="+ date + "&to_date="+ date +
"&time="+ obs_time
```

For at få den præcise position af stjernen skal man give API'en en dato og klokkeslæt som vi får fra en UTC server vi kalder på inden vi sender URL med tid. Efter URL er sen får vi data, som vi bruger til at tracke stjernen med, Det smarte med "astronomyapi"er, at den giver beregninger direkte i altitude og azimut uden, at vi behøver at omregne dataet.

4.3 Position udregning

Hvis man ikke har en API der kan give en de præcise vinkler kan man hente himmellegemets data og beregne vinklerne. Dette kan man gøre på denne måde:

Når vi kender 'right ascension' (RA) og 'declination' til den gældende stjerne, kan vi konvertere til koordinater vores stepper og servo motor kan forstå. Først omregnes RA til hour, minute, seconds (HMS) og declination til degrees, minutes, seconds (dms). Vi henter samtidig epoke tid, som kan konverteres til local sidereal time (LST). Når vi kender RA i grader og LST, kan vi regne hour angle (HA), som fortæller os, om hvorvidt at stjernen er på øst eller vest siden af meridianen. Når HA kendes kan det bruges til cosinus- og sinusrelation til at beregne en azimut og altitude. Disse to værdier giver vi til servo og finder herved stjernens position.

4.3.1 IMU tilt korrektion

Ved at bruge en accelerometer, bruges accelerationen til at bestemme roll og pitch af platformen. ved at inputte roll, pitch og den ønskede elevation og azimut grad, så kan den nødvendige elevation og azimut vinkel beregnes, da det ikke er det samme, hvis platformen står skævt. Dette gøres ved at omforme elevation og azimut til en 3D-vektor og lave en rotationsmatrice fra roll og pitch. Dot-produktet af 3D-vektoren og den inverse rotationsmatrice giver den oprettede 3D-vektor, som kan laves tilbage til elevation og azimut vinkel. Udledningen af matematikken bag kan findes i appendix.

4.4 Manual mode

Hvis brugeren selv vil styre kameraet, så er det også muligt. Hvis brugeren trykker ned på joysticket, skifter den til manuel mode, og derfra er det muligt at styre systemet til lige den position de ønsker. Når de er færdige med at være i manuel mode så trykker de igen joysticket ned og den vil skifte til normal mode.

4.5 Stepper and Servo funktioner

For at følge stjernerne på himlen skal man bruge præcision, der er derfor blevet valgt at bruge en servo og en steppermotor, da disse begge er vinkelstyret og præcise. Præcision er vigtigt, vi har derfor valgt at bruge servo til altitude, da vi så har 90 steps mellem vandret og lodret, mens vi bruger stepper til azimut. Gearingen mellem stepper og tracker gør, at

vi får en præcision på 0.42°. Da en ESP32 bliver brugt til at køre både servo og stepper uden to forskellige strømforsyninger, bliver man nødt til at limitere, hvor meget strøm de trækker, da både servo og stepper er nogle strømsultne komponenter. Der er derfor blevet skrevet speciel kode til at køre en af dem ad gangen og kunne styre deres hastighed, de kører med, ved at bestemme step-delayet, altså tiden mellem hvert step. Ved at styre delay mellem hvert step kan vi sørge for, at det er en glat acceleration og stop, hvilket gør strømforbruget lavere og nemmere for ESP32-boardet at køre. Vi kan derfor slippe afsted med en laveffekts-strømforsyning, hvilket gør, at batterier holder længere, og at der ikke er en kæmpe, klodset strømforsyning. Denne tracker kan køres på en lille powerbank, hvilket gør den transportabel.

5 Opsumering og reflektion af prototype resultater

Generelt set er vi meget tilfredse med, hvor langt vi er kommet med vores prototype. Systemet virker og kan vise forskellige stjernebilleder på himlen, samt brugeren har mulighed for at styre den manuelt.

Det nuværende design bærer præg af at være en prototype. Dette kommer i form af breakout boards, hængende ledninger og udsatte tandhjul. Vores forventninger er dog blevet mødt angående prototypens "parathed", da dette foregik over 3 uger.

Det ville klæde systemet at få printet nogle PCB'er, da dette kan gøre formfaktoren mere kompakt. En endelig model ville have ledningskanaler for at skjule de mange ledninger mellem komponenter og ESP32. Disse ændringer ville derved give et mere kompakt og poleret produkt.

Hjernen bag prototypen (ESP32) er i denne prototype forsynet af en USB-ledning koblet til en computer. Dette ville skulle blive ændret i fremtidige prototyper. For at undgå tilkoblingen af en computer vil vi bruge et batteri eller solceller kombineret med et internt op-/afladningssystem. Hermed ville man kunne gøre prototypen i stand til at være en håndholdt gadget.

Lige nu har vi en begrænsning på præcision i forhold til altituden, på grund af at servomotoren kun tager heltalsværdier. Hvilket betyder, hvis den får en halv grad (0.5°), så vil den runde op til nærmeste heltal.

For at imødekomme denne præcisionsgrænse ville en dyrere og mere præcis servo eller stepper motor blive brugt. Dette ville gøre stjernetrackeren mere præcis, selv over lang tid og små vinkelændringer.

I fremtiden ville en optimering være at implementere solceller på systemet. Det ville gøre det muligt at opstille systemer i parker, skoler og ude i aktivitets områder, uden behov for ekstern strømkilde. Samtidig ville det være en mulighed for at opnå et bæredygtigt system som ville støtte FN's 7. og 11. verdensmål vedrørende bæredygtig energi og bæredygtige byer.

For at prototypen skal være håndholdt eller nemt kalibreres, ville et GPS modul være nødvendigt, da denne kan finde ens præcise lokation. Med denne GPS ville det være muligt at indstille Stjerne trackeren lige hvor man har lyst, uden anden kalibrering.

Vores Prototype planner kan ses i appendix4.

6 Appendix

Bill of Materials

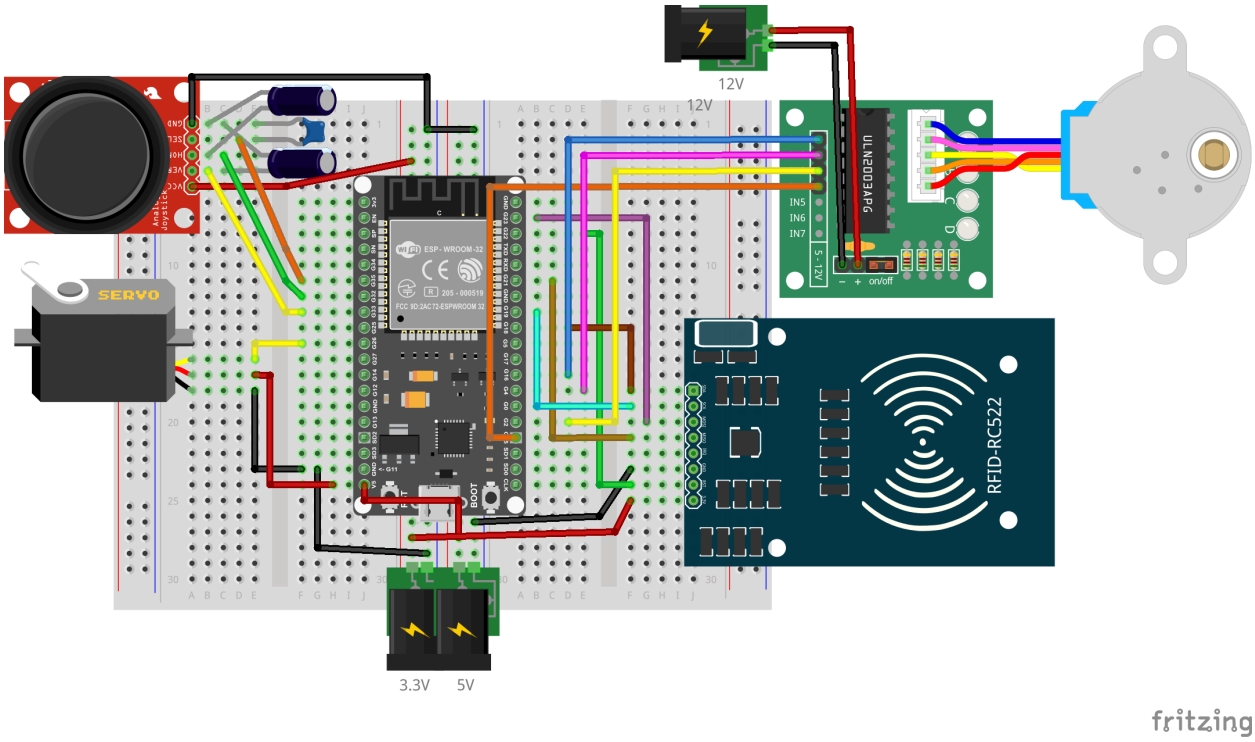
Bill of Materials					
Part name	**Qty**	**Mass [g]**	**Material**	**Manufacturing process**	**Cost [DKK]**
Ceramic Capacitor	1	1	Ceramic/Plastic	Mass production	0,5
Electrolytic Capacitor	2	2	Metal/Plastic	Mass production	1
28BYJ-48 Driver Module	1	12	PCB/Plastic	PCB assembly	15
Basic Servo	1	9	Plastic/Metal	Assembly	15
RFID-RC522-v3	1	5	PCB/Plastic	PCB assembly	25
Thumb Joystick	1	8	Plastic/Metal	Assembly	8
Power plug	3	4	Metal/Plastic	Injection molding	2
28BYJ-48 Stepper Motor	1	35	Metal/Plastic	Assembly	79
ESP32-38PinWide	1	8	PCB/Plastic	PCB assembly	75
MPU 6500	1	3	PCB/Plastic	PCB assembly	20

Tabel 1: Oversigt over estimerede komponenter til prototypen

IMU matematik

Rotationsmatricen og vektor transformationen: $R = \begin{bmatrix} \cos(\theta) & \sin(\theta)\sin(\phi) & \sin(\theta)\cos(\phi) \\ 0 & \cos(\phi) & -\sin(\phi) \\ -\sin(\theta) & \cos(\theta)\sin(\phi) & \cos(\theta)\cos(\phi) \end{bmatrix}$ og $\vec{V}_{world} = \begin{bmatrix} \cos(\epsilon)\cos(\alpha) \\ \cos(\epsilon)\sin(\alpha) \\ \sin(\epsilon) \end{bmatrix}$ Derefter kan den nye vektor findes til: $\vec{V}_{komp} = R^{-1} \cdot \vec{V}_{world}$. Den kan så dannes tilbage til azimut og elevation ved: $\epsilon_{ny} = \arctan\left(z, \sqrt{x^2 + y^2}\right)$ og $\alpha_{ny} = \arctan(y, x)$.

Fritzing diagram



Figur 3: Oversigt over elektriske komponenter

PROTOTYPING PLANNER

PROTOTYPE OF: Star Camera | PROTOTYPE BY: Group 16 | DATE: 09-05-25 | VERSION: 1.0

THINK

PURPOSE

Prototype for X:

○ Human desirability ☒ Technical feasibility ○ Business viability

Why are you prototyping? What are you testing? State the question that your prototype should answer.

Prototyping a arm that moves in 2D, level correction and the ability to find stars according to data pulled from internet.

DESIRED OUTCOME

What is your main objective?

☒ Exploration ☒ Evaluation ○ Communication

What do you hope to achieve? What will you be able to do after prototyping?

We explore if the prototype is able to find stars and if so, evaluate improvements for next version.

FOCUS

Which part of your idea do you wish to prototype? Which concepts are you testing?

3 Focuses:
Mechanical ability to move camera
Electronics: Being able to get accelerometer data and to communicate with internet.
Math: convert acceleration to desired elevation and azimuth.

BUILD

WHAT TO BUILD

Level of detail and realism needed:

Low (quick and dirty) 1 2 ☒ 4 5 High (precise but slow)

What must the prototype be able to do or show? What are its minimum requirements?

The prototype should demonstrate the ability to point at a star with a reasonable accuracy.

BUILD PLAN

How will you make the prototype? Which resources do you need? Which limitations exist?

The Mechanical parts are 3D Printed. A Servo, Steppermotor, ESP32, IMU, Joystick, RFID chip and card and wifi connection is needed.

TEST

TEST PLAN

Who will see or use the prototype? How will you test the prototype?

The prototype will be presented at a demonstration at 09/05/2025
The test will demonstrate the ability to find a star with use of RFID.
We test the ability to move with joystick.

RESULTS

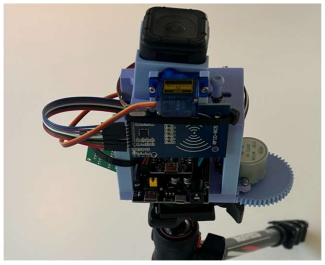
What data will you collect from the test? How? When is the test successful? Define acceptance criteria.

We will take pictures with a mounted gopro, and verify with a star finder app. If there are clear skies the picture will be directly analyzed.
Compass and protractor are also used.

PROTOTYPE

Build and test the prototype. Show it here.

Prototype mounted on tripod.



ACT

INSIGHTS (after prototyping)

Which insights and ideas did the prototyping give you?

A IMU that can recognize North would help with the automatic alignment.
The base was slipping when moving on slippery surfaces.
Low torque from steppermotor.
Too big steps from servo.

ACTIONS

Which conclusions and decisions can you make from these insights?

A base with a better footing should be made. Better packaging should be created, which would hide electronics and also improve waterproofing for outdoor photographing. Lower gearing for steppermotor and a more precise servo.

EVALUATE TEST

Effort: 1 2 3 4 ☒ (Low – High)

Success: 1 2 3 4 ☒ (Low – High)

Is further testing needed? What changes will you make for the next test?

From the protractor and compass it was found to be accurate. The timelapse of a clear skies show good ability to track the moon.

Figur 4: Prototype Planner

7 Arbejdsfordeling

Niels	IMU, Altitude/azimuth angle calc, Joystick
Janus	IMU, Altitude/azimuth angle calc, Joystck
Oscar	API, Wifi, NFC, Altitude/azimuth angle calc
Liou	API, wifi, NFC, Altitude/azimuth angle calc
Elias	CAD, test
Jacob	Design og video
Marcus	Motor/servo, API, Joystick, NFC, Assembly, 3D print, main code

Tabel 2: Arbejdsfordeling

8 Code

Listing 1: IMU.cpp

```
#include "FastIMU.h"
#include <Wire.h>
#include <math.h>
#include "IMU.hpp"

int setupIMU(){
    Wire.begin(); //Initialize I2C
    Wire.setClock(400000); //400khz clock
    int err = IMU.init(calib, IMU_ADDRESS);
    if (err != 0) {
        return err; //Return error if IMU initialization fails
    }
    IMU.calibrateAccelGyro(&calib); //Calibrate the IMU
    IMU.init(calib, IMU_ADDRESS); //Initialize the IMU with the
        calibration data.
    if (err != 0) {
        return err; //Return error if IMU initialization fails
    }
    return 0; //Return 0 if initialization is successful
}


adjustedData adjust(float targetElevation, float targetAzimuth) {
    adjustedData result;
    IMU.update();
    IMU.getAccel(&accelData);
    Xg = accelData.accelX;
    Yg = accelData.accelY;
    Zg = accelData.accelZ;

    //Low Pass Filter
    fXg = Xg * alpha + (fXg * (1.0 - alpha));
    fYg = Yg * alpha + (fYg * (1.0 - alpha));
    fZg = Zg * alpha + (fZg * (1.0 - alpha));

    pitch = atan2(fXg,fZg)* (180/3.14);
    roll = atan2(fYg, fZg)* (180/3.14);

    // get the compensated azimuth and elevation angles and store
        them in the result struct and return it.
```

```
    getCompensatedAzEl(pitch, roll, targetAzimuth, targetElevation
        , result.azimuth, result.elevation);
    return result;

}


//function to convert pitch and roll to a 3D vector
Vec3 rotationMatrixTransform(Vec3 v, float pitch, float roll) {
    // converts to radians
    float pitchRad = pitch * DEG_TO_RAD;
    float rollRad = roll * DEG_TO_RAD;

    // rotates around X (pitch)
    // v1 is the vector after rotation around X
    Vec3 v1;
    v1.x = v.x;
    v1.y = cos(pitchRad) * v.y - sin(pitchRad) * v.z;
    v1.z = sin(pitchRad) * v.y + cos(pitchRad) * v.z;

    // Thereafter rotates around Y (roll)
    // v2 is the vector after rotation around X and Y
    Vec3 v2;
    v2.x = cos(rollRad) * v1.x + sin(rollRad) * v1.z;
    v2.y = v1.y;
    v2.z = -sin(rollRad) * v1.x + cos(rollRad) * v1.z;

    return v2;
  }

// Convert azimuth and elevation angles to a 3D vector
Vec3 azElToVector(float azimuthDeg, float elevationDeg) {
    float az = azimuthDeg * DEG_TO_RAD;
    float el = elevationDeg * DEG_TO_RAD;

    Vec3 v;
    v.x = cos(el) * cos(az);
    v.y = cos(el) * sin(az);
    v.z = sin(el);
    return v;
  }

// Convert the 3D vector back to azimuth and elevation angles
void vectorToAzEl(Vec3 v, float &azimuth, float &elevation) {
```

```
    elevation = atan2(v.z, sqrt(v.x * v.x + v.y * v.y)) *
        RAD_TO_DEG;
    azimuth = atan2(v.y, v.x) * RAD_TO_DEG;
    if (azimuth < 0) azimuth += 360;
  }

// Function that converts target azimuth and elevation angles to
    vector coordinates
// and then applies the inverse rotation to get the adjusted
    Vector coordinates
// Finally, it converts the adjusted vector back to azimuth and
    elevation angles
void getCompensatedAzEl(float pitch, float roll, float targetAz,
    float targetEl, float &adjAz, float &adjEl) {
    Vec3 targetWorld = azElToVector(targetAz, targetEl);

    // Inverse rotation: apply the negative of the pitch and roll
        angles
    // to the target vector to get the adjusted vector
    Vec3 targetLocal = rotationMatrixTransform(targetWorld, -pitch
        , -roll);

    vectorToAzEl(targetLocal, adjAz, adjEl);
  }
```

Listing 2: IMU.hpp

```
#include "FastIMU.h"
#include <Wire.h>
#include <math.h>


struct Vec3 {
    float x;
    float y;
    float z;
};

struct adjustedData {
    float elevation;
    float azimuth;
};

const float alpha = 0.5;
float pitch, Xg, Yg, Zg;
```

```
float fXg = 0;
float fYg = 0;
float fZg = 0;
float roll=0;

#define IMU_ADDRESS 0x68      //Change to the address of the IMU
#define PERFORM_CALIBRATION //Comment to disable startup
    calibration
MPU6500 IMU;                  //Change to the name of any supported
    IMU!


calData calib = { 0 };  //Calibration data
AccelData accelData;    //Sensor data
GyroData gyroData;
MagData magData;


int setupIMU();


adjustedData adjust(float targetElevation, float targetAzimuth);


//function to convert pitch and roll to a 3D vector
//Vec3 rotationMatrixTransform(Vec3 v, float pitch, float roll);

// Convert azimuth and elevation angles to a 3D vector
//Vec3 azElToVector(float azimuthDeg, float elevationDeg);

// Convert the 3D vector back to azimuth and elevation angles
//void vectorToAzEl(Vec3 v, float &azimuth, float &elevation);

// Function that converts target azimuth and elevation angles to
    vector coordinates
// and then applies the inverse rotation to get the adjusted
    Vector coordinates
// Finally, it converts the adjusted vector back to azimuth and
    elevation angles
//void getCompensatedAzEl(float pitch, float roll, float targetAz,
     float targetEl, float &adjAz, float &adjEl) ;
```

Main code

```
    // include the Servo library
#include <Servo.h>
#include <Arduino.h>
#include <SPI.h>
#include <WiFiUdp.h>
#include <WiFiClientSecure.h>
#include <Stepper.h>

#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <NTPClient.h>
#include <MFRC522.h>
#include <base64.h>

const char* ssid =     "***************";
const char* password = "***************";

// AstronomyAPI credentials
const char* applicationId = "***************";
const char* applicationSecret = "***************";

WiFiClientSecure client;
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 2 * 3600, 60000); //
   UTC+2 (sommertid)

void wifisetup() {
// while (!Serial) { ; }
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);  // attempt to connect
  Serial.println();
  Serial.println(F("Wait for WiFi.. "));

  while (WiFi.status() != WL_CONNECTED) {  //wait to connect
    delay(500);
    Serial.print(".");
  }

  Serial.println("WiFi connected");
}
```

```
//(K benhavn-ish)
const float latitude = 55.6761;
const float longitude = 12.5683;
const int elevation = 14;

float targetAltDeg = 0.0;
float targetAzDeg = 0.0;

// dato (format: YYYY-MM-DD)
String getCurrentDate() {
  time_t rawTime = timeClient.getEpochTime();
  struct tm* timeinfo = localtime(&rawTime);

  char buf[11];
  strftime(buf, sizeof(buf), "%Y-%m-%d", timeinfo);
  return String(buf);
}

// tid (format: HH:MM:SS)
String getCurrentTime() {
  time_t rawTime = timeClient.getEpochTime();
  struct tm* timeinfo = localtime(&rawTime);

  char buf[9];
  strftime(buf, sizeof(buf), "%H:%M:%S", timeinfo);
  return String(buf);
}


// Hovedfunktion der henter stjernedata
void getStarInfo(const char* starName) {
  // Opdater tid
  timeClient.forceUpdate();

  String date = getCurrentDate();
  String obs_time = getCurrentTime();

  // Lav base64 auth string
  String credentials = String(applicationId) + ":" + String(
     applicationSecret);
  String encodedAuth = base64::encode(credentials);
  String authHeader = "Basic " + encodedAuth;

  // Lav URL med dynamisk stjernenavn og tidspunkt
```

```
String url = "https://api.astronomyapi.com/api/v2/bodies/
    positions/";
url += starName;
url += "?latitude=" + String(latitude);
url += "&longitude=" + String(longitude);
url += "&elevation=" + String(elevation);
url += "&from_date=" + date;
url += "&to_date=" + date;
url += "&time=" + obs_time;

Serial.println("        Henter data fra:");
Serial.println(url);

// Lav HTTPS GET request
WiFiClientSecure client;
client.setInsecure();

HTTPClient https;
https.begin(client, url);
https.addHeader("Authorization", authHeader);

int httpCode = https.GET();

if (httpCode > 0) {
  Serial.printf("    HTTP-kode: %d\n", httpCode);
  String payload = https.getString();
  // Serial.println("        Svar:");
  // Serial.println(payload);

  DynamicJsonDocument doc(2048);
  DeserializationError err = deserializeJson(doc, payload);

  if (!err) {
    JsonObject root = doc["data"]["table"]["rows"][0]["cells
        "][0]["position"]["horizontal"];
    targetAltDeg = root["altitude"]["degrees"].as<float>();
    targetAzDeg = root["azimuth"]["degrees"].as<float>();

    Serial.printf("        Altitude: %.2f  \n", targetAltDeg);
    Serial.printf("        Azimuth: %.2f  \n", targetAzDeg);
    doc.clear();
    payload.clear();
  } else {
    Serial.print("    JSON-fejl: ");
```

```
      Serial.println(err.c_str());
    }
  } else {
    Serial.printf("    HTTP-fejl: %s\n", https.errorToString(
       httpCode).c_str());
  }

  https.end();
}
```

```
// bruger ikke (                )
/*
float hmsToDegrees(const char* hms) {
  int h, m;
  float s;
  sscanf(hms, "%dh %dm %fs", &h, &m, &s);
  return (h + m / 60.0 + s / 3600.0) * 15.0;  // 24h = 360
}

// a converter of degrees, minute, seconds to degrees. Should be
   used for declination.
// input should be +38deg, 46' 58.8" since the sscanf will convert
    this... This is a c-style string parsing found on
   stackoverflow..
float dmsToDegrees(const char* dms) {
  int d, m;
  float s;
  char sign = dms[0];
  sscanf(dms + 1, "%d  %d  %f  ", &d, &m, &s);
  float deg = d + m / 60.0 + s / 3600.0;
  return (sign == '-') ? -deg : deg;
}

// JD = julian date, 86400 seconds in a day, and you add
   2440587.5, due to how julian date is calculated
// julian date is from 4713 f.v.t to 1970 (epoch time), which is
   the 2440587 in days.
float getLocalSiderealTime(float lon, unsigned long unixTime) {
```

```
  float JD = unixTime / 86400.0 + 2440587.5;
  float D = JD - 2451545.0;
     //this is JD - January 1st 2000, according to julian day.
  float GMST = fmod(18.697374558 + 24.06570982441908 * D, 24.0);
     //gmst is the Greenwich mean sidereal time
  float LST = fmod(GMST + lon / 15.0, 24.0);
     // lst is the local sidereal time..
  if (LST < 0) LST += 24.0;
  return LST * 15.0;  // convert the return in degrees, due to
     conversion with 15 hours.
}

void calculateAltAz(float raDeg, float decDeg, float latDeg, float
    lonDeg, unsigned long unixTime) {
  float lstDeg = getLocalSiderealTime(lonDeg, unixTime);
  float haDeg = lstDeg - raDeg;
  if (haDeg < 0) haDeg += 360;

  float haRad = radians(haDeg);
  float decRad = radians(decDeg);
  float latRad = radians(latDeg);

  float sinAlt = sin(decRad) * sin(latRad) + cos(decRad) * cos(
     latRad) * cos(haRad);
  float alt = asin(sinAlt);

  float cosAz = (sin(decRad) - sin(alt) * sin(latRad)) / (cos(alt)
      * cos(latRad));
  float az = acos(cosAz);
  if (sin(haRad) > 0) az = 2 * PI - az;

  targetAltDeg = degrees(alt); // gem til brug i getrfid
  targetAzDeg = degrees(az);

  Serial.printf("Altitude: %.2f  , Azimuth: %.2f  \n",
     targetAltDeg, targetAzDeg);
}
*/



// Stepper servo funktioner ^_________________^
```

```
bool trackingTicker = false;
String trackingStarName = "";
unsigned long lastTrackUpdateTime = 0;
const unsigned long trackingInterval = 60000; // 1 minutter i
   millisekunder

Servo myServo;  // create a servo object

int const ServoPin = 26;

int sensorPin1 = 32;   // select the input pin for the
   potentiometer
int sensorPin2 = 33;

int ledPin = 2;      // select the pin for the LED
int sensorValue1 = 0;  // variable to store the value coming from
   the sensor
int sensorValue2 = 0;

#define SS_PIN  5  // ESP32 pin GPIO5
#define RST_PIN 22 // ESP32 pin GPIO22

MFRC522 rfid(SS_PIN, RST_PIN);

void rfidsetup() {
  SPI.begin(); // init SPI bus
  rfid.PCD_Init(); // init MFRC522
  Serial.println("Tap an RFID/NFC tag on the RFID-RC522 reader");
}

//! RFID Variables
byte starKey1[4] = { 0x04, 0xF4, 0xD5, 0xB5 };  // Lious rejse
   kort 1
byte starKey2[4] = { 0xF3, 0x9A, 0x1F, 0x42 };  // Liou rejse kort
    2
byte starKey3[4] = { 0x94, 0xC2, 0x0A, 0x83 };  // Elias
   studiekort
byte starKey4[4] = { 0xD4, 0xBF, 0x1E, 0x53 };  // Liou studiekort
byte starKey5[4] = { 0xCD, 0xFC, 0x86, 0xCF };  // Elias rejsekort
byte starKey6[4] = { 0xBA, 0x18, 0x5A, 0x4E };  // Marcus N gle

// Sammenlign to UID arrays
bool compareUID(byte *a, byte *b, byte size = 4) {
```

```
  for (int i = 0; i < size; i++) {
    if (a[i] != b[i]) return false;
  }
  return true;
}

// Return r kort-ID som tal (1-5), eller -1 hvis ukendt
int getCardID(byte *uid) {
  if (compareUID(uid, starKey1)) return 1;
  if (compareUID(uid, starKey2)) return 2;
  if (compareUID(uid, starKey3)) return 3;
  if (compareUID(uid, starKey4)) return 4;
  if (compareUID(uid, starKey5)) return 5;
  if (compareUID(uid, starKey6)) return 6;
  return -1; // Ukendt kort
}

// Get that id my guy (^///^)
void getrfid() {
  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial())
     {
    Serial.print("UID:");
    for (int i = 0; i < rfid.uid.size; i++) {
      Serial.print(rfid.uid.uidByte[i] < 0x10 ? " 0" : " ");
      Serial.print(rfid.uid.uidByte[i], HEX);
    }
    Serial.println();

    int cardID = getCardID(rfid.uid.uidByte);

    switch (cardID) {
      case 1:
        Serial.println("Lious rejsekort1");
        Serial.println("Sun");
        getStarInfo("Sun");
        moveServoSmooth((int)targetAltDeg, 5);
        moveStepperToAngle(targetAzDeg, 3);
        break;
      case 2:
        Serial.println("Lious rejsekort2");
        Serial.println("saturn");
        getStarInfo("saturn");
        moveServoSmooth((int)targetAltDeg, 5);
        moveStepperToAngle(targetAzDeg, 3);
```

```
        break;
      case 3:
        Serial.println("Elias studiekort");
        Serial.println("jupiter");
        getStarInfo("jupiter");
        moveServoSmooth((int)targetAltDeg, 5);
        moveStepperToAngle(targetAzDeg, 3);
        break;
      case 4:
        Serial.println("Liou studiekort");
        Serial.println("mercury");
        getStarInfo("mercury");
        moveServoSmooth((int)targetAltDeg, 5);
        moveStepperToAngle(targetAzDeg, 3);
        break;
      case 5:
        Serial.println("Elias rejsekort");
        Serial.println("moon");
        getStarInfo("moon");
        moveServoSmooth((int)targetAltDeg, 5);
        moveStepperToAngle(targetAzDeg, 3);
        break;
      case 6:
        Serial.println("Elias fitness");
        Serial.println("Track sun");
        trackingStarName = "sun";  // tag navnet efter mellemrum
        trackingTicker = true;
        lastTrackUpdateTime = -600000;  // s   den opdaterer med
           det samme
        Serial.print("Tracking Ticker AKTIVERET for: ");
        Serial.println(trackingStarName);
        break;
      default:
        Serial.println("Ukendt kort");
        break;
    }
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
  }
}

const int manualButtonPin = 35;

// Tilstande
```

```
bool manualMode = false;
bool lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 200;

const int stepsPerRevolution = 2048;          // steps per
   revolution
const int RevolutionsPerMinute = 15;          // Adjustable range
   of 28BYJ-48 stepper is 0~17 rpm

// initialize the stepper library on pins 15, 4, 2, 16:
Stepper myStepper(stepsPerRevolution, 15, 4, 2, 16);

int currentAngle = 0;  // Startposition (kan v re 0-180)

// Funktion der bev ger servoen langsomt til en vinkel
void moveServoSmooth(int targetAngle, int stepDelay) {
  if (targetAngle > 180) targetAngle = 180;
  if (targetAngle < 0) targetAngle = 0;

  int step = (targetAngle > currentAngle) ? 1 : -1;

  for (int angle = currentAngle; angle != targetAngle; angle +=
     step) {
    myServo.write(angle);
    delay(stepDelay);
  }

  myServo.write(targetAngle); // s rg for at lande pr cist
  currentAngle = targetAngle;
}

float stepCurrentAngle = 0.0; // husker hvor vi er (0-360  )

// Funktion: Drejer til en vinkel langsomt
void moveStepperToAngle(float targetAngle, int stepDelay) {
  // wrap rundt om 0-360
  targetAngle = fmod((targetAngle - 360), 360);

  // Beregn hvor mange step der skal k res
  float angleDiff = targetAngle - stepCurrentAngle;

  // s rg for at vi altid v lger den korteste vej rundt
  if (angleDiff > 180) angleDiff -= 360;
```

```
  if (angleDiff < -180) angleDiff += 360;

  int stepsToMove = (int)(angleDiff * (stepsPerRevolution / 360.0)
     );
  int stepDirection = (stepsToMove > 0) ? 1 : -1;

  for (int i = 0; i < abs(stepsToMove); i++) {
    myStepper.step(stepDirection); // bev g 1 step
    delay(stepDelay);              // pause mellem steps
  }

  // Opdater vinkel-tracking
  stepCurrentAngle = targetAngle;
}

String inputString = "";
bool stringComplete = false;

// === Serial input h ndtering ===
void serialEvent() {
  while (Serial.available()) {
    char inChar = (char)Serial.read();
    if (inChar == '\n') {
      stringComplete = true;
    } else {
      inputString += inChar;
    }
  }
}


void servoSetup(){
  myServo.attach(ServoPin);   // attaches the ServoPin to the
     servo object

  moveServoSmooth(90, 5);
  delay(1000);
  moveServoSmooth(0, 5);  // closes servo
  delay(500);

}
```

```
void stepperSetup(){
  myStepper.setSpeed(RevolutionsPerMinute);
  moveStepperToAngle(45, 3);
  delay(1000);
  moveStepperToAngle(0, 3);
  delay(500);
}



void setup() {
  // declare the ledPin as an OUTPUT:
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);  // LED slukket til at starte med
  pinMode(manualButtonPin, INPUT);
  Serial.begin(115200);
  servoSetup();
  stepperSetup();
  rfidsetup();
  wifisetup();
  btStop();
  Serial.println("Skriv et stjernebillede (fx: karlsvogn, orion,
     cassiopeia, pleiaderne, lyren)");
  Serial.println("S  t NFC chip op mod scanner for at f   dets
     stjernebillede");
  Serial.println("Skriv: trackticker **** 'erstat stjerner med dit
      valgte legme, disse legmer er: ");
  Serial.println("sun, moon, mercury, venus, mars, jupiter, saturn
     , uranus, neptune, pluto");
}


// === Skift mellem automatisk og manuel ===
void checkManualButton() {
  int analogValue = analogRead(manualButtonPin); // 0  4095
  bool buttonPressed = analogValue > 3000; // svarer til >3V

  if (buttonPressed != lastButtonState && millis() -
     lastDebounceTime > debounceDelay) {
    lastDebounceTime = millis();

    if (buttonPressed) {
      manualMode = !manualMode;
      digitalWrite(ledPin, manualMode ? HIGH : LOW); // LED t nd/
```

```
        sluk
      Serial.println(manualMode ? "Manuel mode AKTIVERET" : "
        Automatisk mode AKTIVERET. Skriv et stjernebillede (fx:
        karlsvogn, orion, cassiopeia, pleiaderne, lyren)");
    }
  }

  lastButtonState = buttonPressed;
}


// ==== Manuel styring ====
void manualControl() {
  int sensorValue1 = analogRead(sensorPin1);
  int sensorValue2 = analogRead(sensorPin2);

  int servoTarget = map(sensorValue1, 0, 4095, 0, 180);
  float stepperTarget = map(sensorValue2, 0, 4095, 0, 360);

  moveServoSmooth(servoTarget, 2);
  moveStepperToAngle(stepperTarget, 3);
}

// ==== Stjernebillede-funktioner ====
void processStarCommand(String input) {
  input.trim();
  input.toLowerCase();

  if (input.startsWith("trackticker ")) {
    trackingStarName = input.substring(12);  // tag navnet efter
      mellemrum
    trackingTicker = true;
    lastTrackUpdateTime = -600000;  // s   den opdaterer med det
      samme
    Serial.print("Tracking Ticker AKTIVERET for: ");
    Serial.println(trackingStarName);
  } else if (input == "karlsvogn") {
    trackingTicker = false;
    moveServoSmooth(30, 5);
    moveStepperToAngle(45, 3);
  } else if (input == "orion") {
    trackingTicker = false;
    moveServoSmooth(60, 5);
    moveStepperToAngle(110, 3);
```

```
  } else if (input == "cassiopeia") {
    trackingTicker = false;
    moveServoSmooth(80, 5);
    moveStepperToAngle(290, 3);
  } else if (input == "pleiaderne") {
    trackingTicker = false;
    moveServoSmooth(40, 5);
    moveStepperToAngle(120, 3);
  } else if (input == "lyren") {
    trackingTicker = false;
    moveServoSmooth(70, 5);
    moveStepperToAngle(180, 3);
  } else {
    Serial.println("Ukendt kommando.");
  }

  Serial.println("Klar til ny kommando:");
}

// Track that, all on the floor, track that, give me some more,
   track that    *: .    . o(          )o .    .:*

void trackTicker() {
  unsigned long currentMillis = millis();

  if (trackingTicker && trackingStarName != "") {
    if (currentMillis - lastTrackUpdateTime >= trackingInterval) {
      Serial.print("Henter ny position for: ");
      Serial.println(trackingStarName);

      getStarInfo(trackingStarName.c_str());

      // Brug de globale v rdier som getStarInfo s tter:
      float az = constrain(targetAzDeg, 0, 360);
      float alt = constrain(targetAltDeg, 0, 90);

      moveServoSmooth((int)alt, 5);
      moveStepperToAngle(az, 3);

      lastTrackUpdateTime = currentMillis;
    }
  }
}
```

```
void loop() {
  // read the value from the sensor:
  //sensorValue1 = analogRead(sensorPin1);
  //sensorValue2 = analogRead(sensorPin2);

  //moveServoSmooth(map(sensorValue1, 0, 4095, 5, 175), 20);

  //moveStepperToAngle(map(sensorValue2, 0, 4095, 0, 175), 1);

  //Serial.println(sensorValue1);sas
  checkManualButton();

  if (manualMode) {
    manualControl();
  } else {
    getrfid();
    if (trackingTicker) {
      trackTicker();
    } else {
      if (stringComplete) {
        processStarCommand(inputString);
        stringComplete = false;
        inputString = "";
      }
    }
  }
  delay(50);
}
```